

Tema 7: Cifrado simétrico avanzado

Docente : Héctor Xavier Limón Riaño
email: xavier120@hotmail.com

Cifrado autenticado con datos adicionales

AES-GCM

- ▶ Un problema que tienen los MAC es que requieren de su propia llave, además de que requieren de mayor conocimiento de configuración
- ▶ El cifrado autenticado (AE) y el cifrado autenticado con datos adicionales (AEAD) proveen autenticidad, integridad y confidencialidad
- ▶ AEAD también provee autenticidad adicional sobre datos adicionales no cifrados
- ▶ AES-GCM (Galois/Counter Mode) es un modo de operación de AES con AEAD que reemplaza a un MAC

AES-GCM

- ▶ El iv en este modo de operación es de 12 bytes (más adelante se verá porqué)
- ▶ Al estar basado en CTR no requiere padding
- ▶ Es recomendable usarlo en conjunto con un algoritmo de derivación de llave, como Scrypt (cuando se van a almacenar datos en disco) o HKDF (cuando se va a hacer una transmisión remota)
- ▶ Los datos adicionales pueden ser el iv y el salt usado en la derivación de llave
- ▶ Los datos adicionales no se cifran pero deben autenticarse mediante la llave derivada
- ▶ Al terminar el proceso de cifrado de datos se obtiene un **tag** que es similar a un MAC: da autenticación e integridad sobre todo lo cifrado y también sobre los datos adicionales

AES-GCM (cifrar)

```
1 from cryptography.hazmat.backends import default_backend
2 from cryptography.hazmat.primitives.kdf.scrypt import Scrypt
3 import os
4 from cryptography.hazmat.primitives.ciphers import Cipher,
5     algorithms, modes
6
7 #Cifrar, se requiere un password para Scrypt
8 salt = os.urandom(16)
9 kdf = Scrypt(salt=salt, length=32,
10             n=2**14, r=8, p=1,
11             backend=default_backend())
12 key = kdf.derive(password)
13 iv = os.urandom(12)
14 encryptor = Cipher(algorithms.AES(key),
15                   modes.GCM(iv),
16                   backend=default_backend()).encryptor()
```

AES-GCM (cifrar)

```
1 associated_data = iv + salt
2 encryptor.authenticate_additional_data(associated_data)
3 encryptor.update(textoPlano)
4 encryptor.finalize() # necesario para generar tag
5 tag = encryptor.tag # 16 bytes
```

AES-GCM (descifrar)

```
1 #obtener el iv y el salt y generar de nuevo la llave
2 kdf = Scrypt(salt=salt, length=32,
3             n=2**14, r=8, p=1,
4             backend=default_backend())
5 key = kdf.derive(password)
6 # obtener el tag de algun lado
7 decryptor = Cipher(algorithms.AES(key),
8                   modes.GCM(iv, tag),
9                   backend=default_backend()).decryptor()
10 # revisar integridad y autenticidad de datos adicionales
11 associated_data = iv + salt
12 decryptor.authenticate_additional_data(associated_data)
13 texto_plano = decryptor.update(ciphertext)
14 # si hay alteraciones finalize lanza excepcion
15 decryptor.finalize()
```

Ejercicio

- ▶ Implementar un programa para cifrar y descifrar archivos de cualquier tamaño (hay un límite de 64GB) mediante AES-GCM
- ▶ El programa recibe un password (para la derivación de llave), la ruta del archivo de entrada, la ruta del archivo de salida y la operación que se desea ejecutar (cifrar o descifrar)

AES-GCM

- ▶ El modo permite cifrar y autenticar con una sola llave
- ▶ El cifrado y autenticación está integrada, no es necesario preocuparse de cuando hacer cada cosa, a diferencia de MAC que debes recordar que es cifra luego aplica MAC
- ▶ AEAD incluye también autenticación sobre datos que no están cifrados

AEAD

- ▶ Ha habido muchos problemas en la historia de la criptografía derivados de atacantes que toman datos de un contexto para usarlos en otro
- ▶ Este es el caso por ejemplo de los **ataques de repetición**
- ▶ En muchos casos, estos ataques fallarían si se reforzara el contexto de los datos sensibles, para que no puedan ser usados en otros

AEAD

- ▶ En el ejemplo de código se autenticó el iv y salt, **para evitar que estos datos planos sean alterados, pero en realidad se puede autenticar cualquier información**
- ▶ Por ejemplo, se puede autenticar el nombre y timestamp de creación de un archivo, para evitar que se presente como un nuevo archivo en un ataque de repetición
- ▶ Se puede autenticar cualquier meta-información que permita reconocer el contexto
- ▶ Cuando cifres un archivo, piensa con cuidado qué información debe ser autentica, no sólo privada
- ▶ **Entre mejor puedas identificar y proteger el contexto de los datos cifrados, más seguro será tu sistema**

AEAD

- ▶ Es importante notar que las alteraciones a un archivo cifrado sólo pueden ser descubiertas hasta finalizar el descifrado (cuando se llama a finalize), porque hasta entonces se puede determinar la validez del tag
- ▶ Ten cuidado de siempre hacer la comprobación final (llamar a finalize) de lo contrario podrías estar almacenando o procesando datos inválidos
- ▶ Esto en realidad es un problema más complejo de lo que parece ¿Qué tanto tiempo hay que esperar para obtener un tag válido?

AEAD

- ▶ Supón que se está transmitiendo un archivo muy grande, hay que esperar hasta que termine de transmitirse el archivo para obtener el tag y saber si el archivo estaba bien
- ▶ Además, ¿Cómo puedes calcular el final de un canal seguro?
- ▶ En TLS, por ejemplo, cada registro TLS (más o menos un paquete) es cifrado individualmente con AES-GCM teniendo su propio tag individual
- ▶ De esa forma, cualquier alteración maliciosa puede detectarse rápido

AEAD

- ▶ En general, se recomienda segmentar datos grandes cuando se hacen transmisiones, para poder obtener tags de cada pedazo
- ▶ Mediante esta aproximación no se cifran o descifran datos grandes, por lo que no es necesario tener operaciones como update
- ▶ cryptography provee un API simplificado para este caso

AES-GCM

```
1 import os
2 from cryptography.hazmat.primitives.ciphers.aead import AESGCM
3 data = b"a secret message"
4 aad = b"authenticated and unencrypted data"
5 key = AESGCM.generate_key(bit_length=128)
6 aesgcm = AESGCM(key)
7 nonce = os.urandom(12)
8 ct = aesgcm.encrypt(nonce, data, aad)
9 plain = aesgcm.decrypt(nonce, ct, aad)
```

Detalles de seguridad

Manejo de IV

- ▶ Notar que al igual que con el modo CTR, **NUNCA** se debe usar el mismo par de llave e IV (nonce), de lo contrario se puede extraer el keystream (con un ataque de texto escogido por ejemplo)
- ▶ Hay que recordar que CTR parte de un IV (que debería ser aleatorio) y por cada bloque incrementa el IV en 1
- ▶ En algún momento si se sigue incrementando volverá a 0 y alcanzará el valor aleatorio original

Manejo de IV

- ▶ Para datos relativamente pequeños, un IV de 16 bytes es suficiente
- ▶ En un sistema en producción es necesario hacer un análisis de seguridad, para determinar cuánto tiempo se tiene en promedio antes de crear dos keystreams que se traslapan
- ▶ Este cálculo depende de qué tantos datos se planea cifrar con la misma llave

Manejo de IV

- ▶ Hay algunas reglas que se pueden seguir para mitigar el problema de repetición de IVs
- ▶ Por ejemplo, GCM usa IVs de 12 bytes, a los 4 bytes restantes se les agrega un padding de ceros, para tener un contador de 16 bytes
- ▶ Incluso si se elige un nuevo IV que es uno más de un IV anterior, los contadores no se van a traslapar, a menos que se agoten los 16 bytes

Manejo de IV

- ▶ Por las limitaciones de valores de IV, GCM no acepta más de 2^{36} (64GB), dado este tamaño desborda el contador
- ▶ Por razones de seguridad, en GCM el IV nunca debe ser 0
- ▶ Este límite de GCM no es un problema si se segmenta la información a ser cifrada

Manejo de IV

- ▶ Si se segmentan los datos, es muy importante que en cada segmento se use un IV diferente
- ▶ El IV puede ir como cabecera del segmento de datos
- ▶ Lo anterior es válido incluso en transmisiones, el IV no es un dato secreto por lo que no debe ir cifrado, pero si debe ser auténtico, por lo que se puede autenticar con GCM

Manejo de IV

- ▶ Existen algoritmos que ayudan a prevenir el reuso de IVs
- ▶ En general no es bueno tener algoritmos deterministas para generar llaves (pues pueden llevar a una explotación), pero en el caso de IVs son aceptables siempre y cuando no se repita el mismo par de llave e IV

Otros algoritmos AEAD

Introducción

- ▶ Existen otros algoritmos populares AEAD en cryptography:
 - ▶ AES-CCM
 - ▶ ChaCha

AES-CCM

- ▶ Muy similar a AES-GCM
- ▶ Usa CBC para el cifrado en vez de CTR
- ▶ El tag es computado por un método similar pero superior a CBC-MAC
- ▶ Una diferencia importante es que CCM usa IVs de longitud variable: entre 7 y 13 bytes
- ▶ Al usar un IV más pequeño es posible cifrar datos mayores a 64GB, la seguridad del método no se ve comprometida por un IV pequeño, siempre que no se repita el mismo par IV llave

AES-CCM

- ▶ El API es de CCM es básicamente el mismo que para GCM
- ▶ Sin embargo, al estar basado en CBC, **CCM no es fácilmente paralelizable**, como si lo es GCM
- ▶ Esto es muy importante si se desea usar cómputo GPU para tareas criptográficas
- ▶ En cryptography, **CCM no está soportado como modo de operación de AES**, pero tiene su propio módulo con API simplificado para pocos datos, como GCM

AES-CCM

```
1 import os
2 from cryptography.hazmat.primitives.ciphers.aead import AESCCM
3 data = b"a secret message"
4 aad = b"authenticated but unencrypted data"
5 key = AESCCM.generate_key(bit_length=128)
6 aesccm = AESCCM(key)
7 nonce = os.urandom(7)
8 ct = aesccm.encrypt(nonce, data, aad)
9 plain = aesccm.decrypt(nonce, ct, aad)
```

ChaCha

- ▶ El nombre completo es ChaCha20-Poly1305
- ▶ No está basado en AES
- ▶ Diseñado por Daniel J. Bernstein
- ▶ Combina un cifrador de flujo (ChaCha20) con un algoritmo de MAC (Poly1305)
- ▶ No se han encontrado vulnerabilidades en este algoritmo, por lo que se considera seguro
- ▶ Muchos lo ven como un sucesor de RC4, un cifrador de flujo que se usaba antes en TLS y Wi-Fi, pero que tiene diversas vulnerabilidades

ChaCha

- ▶ Algunos en la comunidad de seguridad están preocupados de que la popularidad de AES signifique que si se le encuentra una vulnerabilidad, toda la seguridad criptográfica en internet colapse
- ▶ ChaCha se está estableciendo como una alternativa efectiva a AES
- ▶ Si se encuentra una vulnerabilidad en AES, se tiene disponible un remplazo muy probado y documentado

ChaCha

- ▶ ChaCha20 tiene incluso algunas ventajas sobre AES
- ▶ Es en general más veloz que AES (sin considerar hardware especializado)
- ▶ Es un cifrador de flujo verdadero, a diferencia de AES que es de bloque pero puede ser usado como si fuera de flujo (con modos derivados de CTR)
- ▶ Al igual que AES-GCM, utiliza IVs de 12 bytes

ChaCha

```
1 from cryptography.hazmat.primitives.ciphers.aead import ChaCha20Po
2 import os
3 data = b"a secret message"
4 aad = b"authenticated but unencrypted data"
5 key = ChaCha20Poly1305.generate_key()
6 chacha = ChaCha20Poly1305(key)
7 nonce = os.urandom(12)
8 ct = chacha.encrypt(nonce, data, aad)
9 plain = chacha.decrypt(nonce, ct, aad)
```

Algoritmos AEAD

- ▶ Los tres algoritmos vistos se consideran seguros
- ▶ Los tres se consideran una alternativa mejor a utilizar MAC
- ▶ Siempre que un algoritmo AEAD esté disponible debe elegirse en lugar de MAC

Ejercicio

- ▶ Hacer una comparación de velocidad de cifrado y descifrado de AES-GCM, AES-CCM y ChaCha

Tarea

- ▶ Esta tarea tiene valor triple
- ▶ Consiste en hacer una aplicación que le permita a un cliente subir o descargar archivos de forma segura
- ▶ El profesor implementará el sistema sin ningún tipo de seguridad criptográfica, tu trabajo consiste en agregar todos los elementos de seguridad criptográfica necesarios, los cuales se describen a continuación

Tarea

Elementos de seguridad:

- ▶ Uso de certificados (pueden ser de llaves RSA o EC): sólo el cliente necesita el certificado del servidor (lo tiene desde el inicio), los clientes no están autenticados con un certificado en el servidor sino que necesitan pasar credenciales de usuario una vez se estableció la sesión segura
- ▶ Establecimiento de sesión segura mediante ECDH, sólo se autentica la parte pública del servidor
- ▶ Se requieren dos llaves simétricas: una para enviar y otra para recibir datos, obviamente estas llaves están cruzadas en cliente y servidor. Derivar las llaves con un algoritmo de derivación de llaves

Tarea

- ▶ Cifrado simétrico mediante algún algoritmo AEAD
- ▶ Se tiene que tener mucho cuidado de no repetir IVs, se pueden agregar como encabezado de cada paquete enviado, simplemente hay que autenticarlo
- ▶ Puntos extra: agregar timestamp autenticado, para que el servidor no admita paquetes que tienen más de 2 horas de vida, mitigando un ataque de repetición

Kerberos

Introducción

- ▶ En la actualidad, PKI (public key infrastructure) es ampliamente utilizado para establecer identidad y confianza entre dos entidades
- ▶ Ejemplos: Certificados, firmas digitales, transporte de llaves RSA, DH y ECDH
- ▶ Sin embargo, existen protocolos que permiten establecer identidad y confianza usando sólo cifrado simétrico

Kerberos

- ▶ Es un protocolo muy conocido basado en cifrado simétrico para autenticación y establecimiento de confianza entre dos entidades que se comunican
- ▶ Es una herramienta relativamente antigua, siendo su versión más actual (versión 5) desarrollada a principios de los 90's, aunque se ha mantenido actualizada

Kerberos

- ▶ Kerberos esencialmente es un servicio que funciona como intermediario entre dos entidades que se desean comunicar, siendo el servicio un tercero de confianza (trusted third party)
- ▶ Un cliente se autentica primero al servicio de Kerberos y esta autenticación le permite tener acceso a los servicios que estén conectados al servicio de Kerberos, sin necesidad de volverse a autenticar

Kerberos

- ▶ La arquitectura de Kerberos tiene dos componentes básicos:
Servidor de autenticación, Servidor de tickets
- ▶ En el siguiente vídeo se explica brevemente el funcionamiento de Kerberos:
https://www.youtube.com/watch?v=_44CHD3Vx-0

Kerberos

- ▶ Como pudo verse, Kerberos es sobre todo de utilidad en protocolos de comunicación que no tienen ninguna seguridad por defecto (TCP/IP en general)
- ▶ Se puede combinar con LDAP, para proveer también autorización
- ▶ Puede requerir de mucha configuración, incluyendo la "Kerberización" de servicios, para que acepten tokens de autenticación, pero muchos servicios suelen tener soporte directo